

DataLakeHouse Architecture Proposal

By Anando Zaman
June 13, 2026

System Requirements

- The system must be able to segregate Data Access between different Business Units for independent management/usage
- The system must be able to support Batch, Realtime, MicroBatch pipeline orchestration due to potential business latency/availability needs (Daily aggregated BI reports, real-time financial transactions monitoring, etc)
- The system must be able to provide a platform to run independent Analytics or ML training workstreams by DS/MLEng/Analysts at the company
- The system must follow a CI/CD patterns with strict tests(unit/integration) and be able to revert back without impacting production
- The system data must be auditable while still keeping data costs low (move to cold/archive storage if inactive)

Architecture Proposal

- **For Pipelining**, we can implement a **Kappa Data Architecture**
 - Streaming and Batch/MicroBatch Pipelines as independent paths
- **For Data Lakehouse**, leverage **Data Medallion Architecture**:
 - Ensures Bronze(Raw), Silver(Clean/Curated), Gold (BI/ML Usecase) datasets follow a lineage and are backfillable (Silver/Gold).
- **For Compute (Processing & Analytics)**, leverage **Azure DataBricks**:
 - Allows to run Spark Workflows (Batch and continuous jobs) orchestration
 - Provides a platform for Analysts/BI/DS/ML folks to build reports/analysis/ML_Training
- For **BU data segregation**, use Azure DatalakeHouse (ADLS) ACLs & Databricks Unity Catalog Roles/ACLs for user-level segregation.
 - Prevents unauthorized tools from dumping data to ADLS.
 - Ensures users w/ roles & permissions can access the data via Unity Catalog.
- **For CI/CD**, leverage Green/Blue Pattern w/ Github Actions & Terraform
 - Ensures we can rollback without production impacts. Includes PyTest

Compute & Storage: What Azure & DataBricks Services Used?

Azure:

Azure Data Lake Storage Gen2 (ADLS Gen2): Storage layer housing physical files for Data Medallion layers.

Azure Event Hubs Central **real-time ingestion engine (Kafka-compatible message broker)** that handles native data pushes from external sources as an immutable log. Data retained a while before being purged.

Event Hubs Capture: A native feature of **Event Hubs** that **automatically micro-batches** and exports the raw incoming streams into cold storage. Necessary for retention & replayability for audit trail or backfills.

Azure Data Factory (ADF): The primary **orchestration tool** for the **Batch Path**, used to securely pull data from relational databases, APIs, and legacy files on a schedule.

Azure Storage Blob SDK / AzCopy: In case Data Factory is not used, can adhoc load data this way to ADLS.

Databricks:

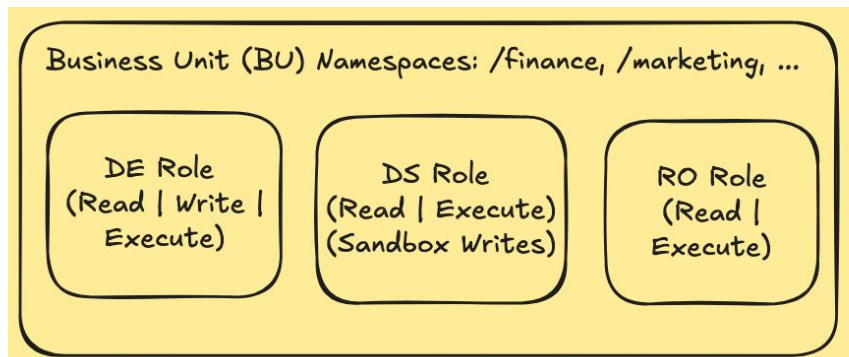
Databricks Continuous Jobs (Realtime Streaming needs): Spins up a cost-optimized Job Cluster and forces a streaming script to run in an infinite loop, providing sub-second processing.

Databricks Workflows (Orchestration Jobs & Schedules): Schedules and chain together the batch processing transformation tasks (Bronze → Silver→Gold).

Databricks SQL / Serverless Warehouses: The compute resource optimized for BI tools and analyst queries to directly hit the Gold layer tables.

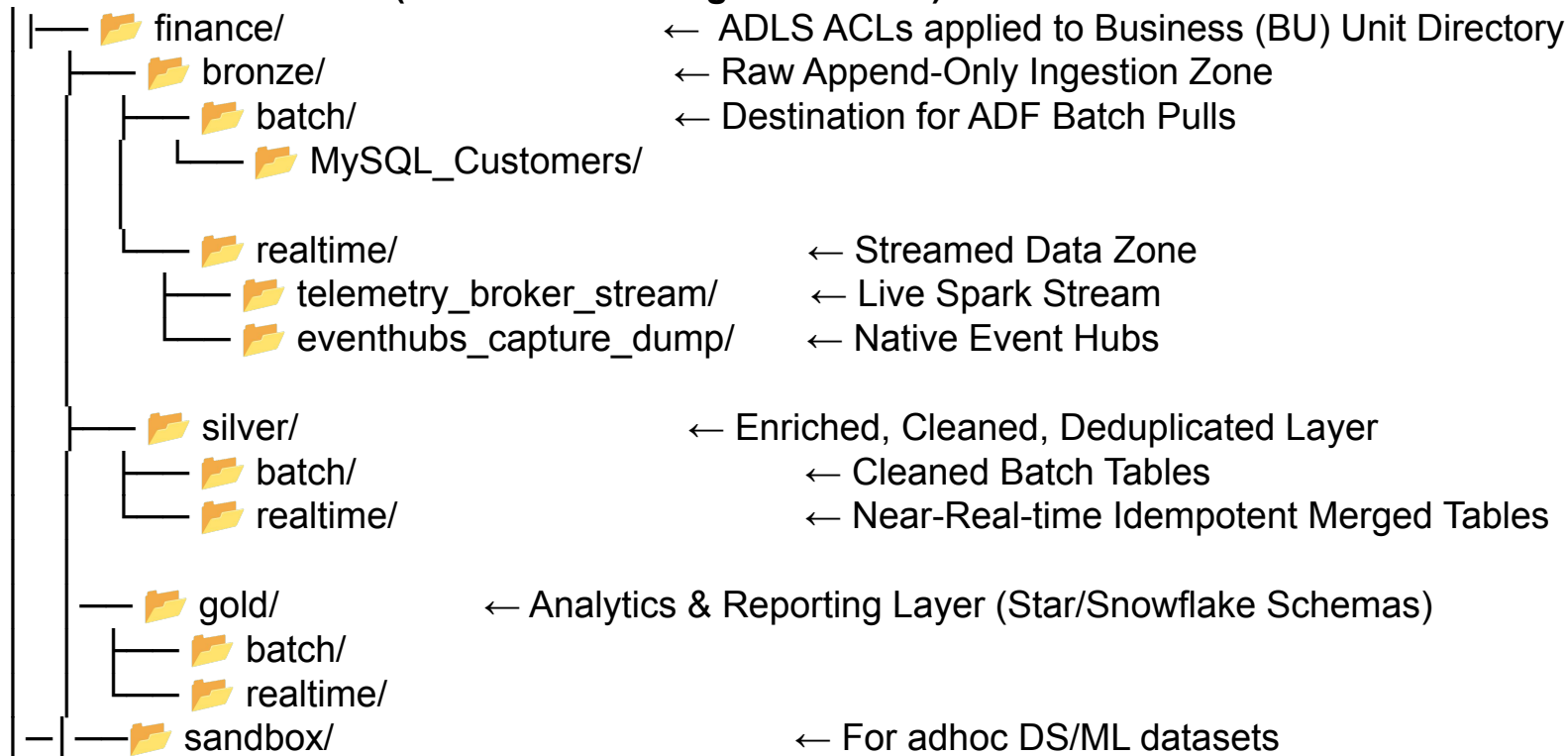
Storage Segregation - Multi-tenant Storage Hierarchy Pattern

- Partitions and isolates data for multiple Business Units (BU) within shared cloud infrastructure relying on ACL enforcement and roles per directory.
 - **ADLS POSIX ACLs on root BU folders:** protect raw directory data and give external tools like ADF and Event Hubs a secure landing spot
 - **Unity Catalog Roles:** Table permission access to MLE/DS/DE/Analysts without worrying about physical folder paths or breaking ADLS permissions.
 - If access is granted by the Catalog, Databricks performs credential vending to query the ADLS location despite the user not having actual ADLS perms.
 - Additionally, Provides PII masking & Lineage too.
- **[Unity Catalog]: Three roles assigned for users in each BU directory**



Storage Segregation - Multi-tenant Storage Folder Structure

lakehouse-container/ (Root Azure Storage Container)



Data Governance & Observability

Observability, Quality & Governance

Unity Catalog

- Data Metastore
- Table Level lineage
- Column Level Lineage

Delta Lake Engine

- ACID Transactions
- Data Versioning / Time Travel

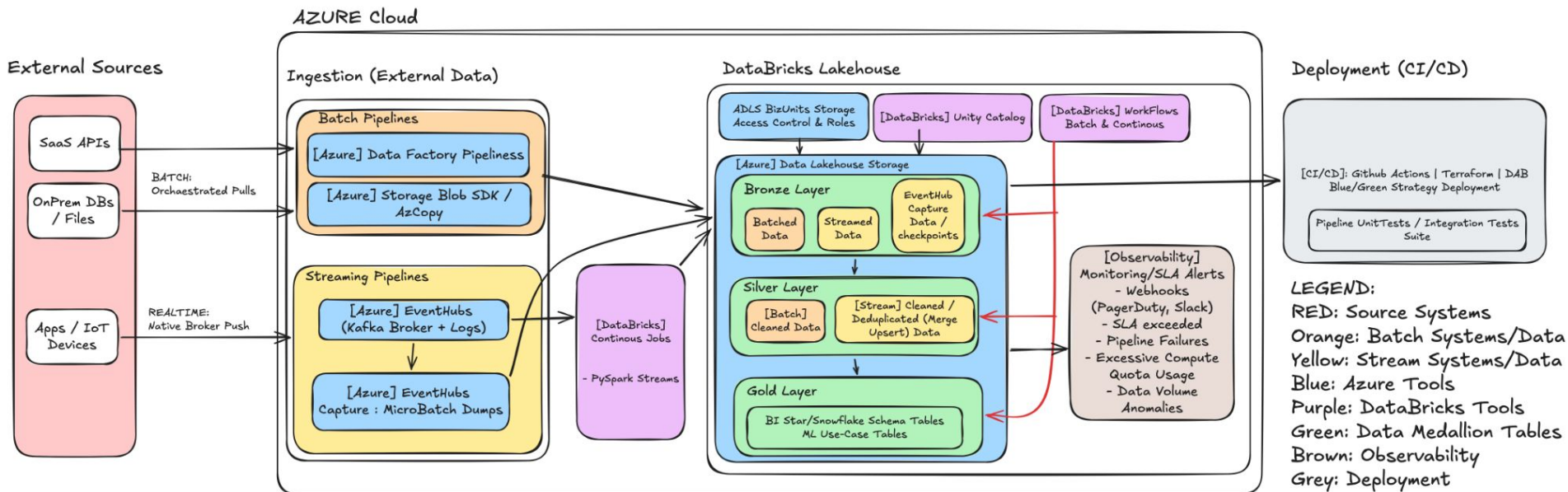
Monitoring/SLA Alerts

- Webhooks (PagerDuty, Slack)
 - SLA exceeded
 - Pipeline Failures
- Excessive Compute Quota Usage
- Data Volume Anomalies

Deployment

- **Infrastructure Management: Terraform**
 - Manage environments (Dev, Test, Prod) and their physical azure resource groups, storage accounts, etc.
 - Separate environments via environment variable files (`dev.tfvars`, `test.tfvars`, `prod.tfvars`)
- **Pipeline Layer (Databricks Asset Bundles (DABs) databricks.yml):**
 - Manages notebook configurations, cluster dependencies, and workflows checked into Git.
 - Takes the notebooks, bundles them up, and configures the exact pipelines inside the workspace that Terraform built.
- **GitHub Actions:**
 - Listens for Git merges, runs test suits, and executes terraform deployment commands
- **The Deployment Strategy: Blue/Green Deployment**
 - **Isolation:** When Git merge/push completes, Github Actions utilizes Databricks DABs yaml to create shadow schemas.
 - **Validation:** The shadow pipeline automatically performs comparisons and data quality checks (via PyTest and Delta Live Table Expectations).
 - **Atomic cutover:** If all looks good, the production table view is replaced to point to the new shadow table table. The old table becomes a backup.

Architecture Diagram



System Trade-offs & Future Evolution

- **System Tradeoffs:**
 - **Compute Costs vs. Sub-Second Latency:**
 - **Trade-off:** Keeping a **Databricks Continuous Job running 24/7** on an always-on cluster guarantees sub-second processing but means we will have **persistent compute costs**.
 - **Potential Change/Mitigation:** If business latency requirements loosen (eg; run 15 mins, hourly, etc), we can instantly switch to an **orchestrated micro-batch pattern** reading from **Event Hubs Capture using Auto Loader** on a 15-minute interval.
- **Future Evolution/Changes:**
 - **Feature Store Integration & Automated Retraining Loops:** Bridge the gap between data engineering and machine learning by formalizing the Databricks Feature Store. When data lands in the Gold layer configure automated triggers via MLflow to evaluate model drift. If drift exceeds threshold limits, GitHub Actions can automatically spin up a training pipeline job, log the new model artifact, and update the serving endpoint.
 - **Develop domain specific AI Agents:** To answer specific questions that the general-purpose Databricks AI is unable to answer with accuracy. Reduced chance of hallucinated responses.